

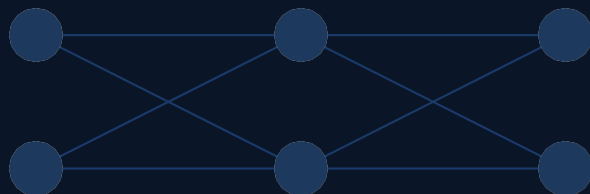
Solving Two Array Problems

Author: Moustafa Fathalla Omar

Topic: solve Array problem

Level: Advanced Machine Learning – problem solving

Year: 2026 / 2027



These two array manipulation problems demonstrate important programming patterns that appear frequently in algorithm interviews and are also relevant to understanding how neural network computations are structured internally.

Problem A — Product of Array Except Self

Task: Given array `nums`, return array where `answer[i] = product of all elements except nums[i]`. No division allowed. Must run in $O(n)$.

Numerical Example

Index	nums[i]	Left Product	Right Product	answer[i]
0	1	1	24	24
1	2	1	12	12
2	3	2	4	8
3	4	6	1	6

Solution 1 — Two extra arrays $O(n)$ space:

```
def product_except_self_v1(nums):
    n = len(nums)

    left = [1] * n    # left[i] = product of nums[0..i-1]
    right = [1] * n   # right[i] = product of nums[i+1..n-1]
    for i in range(1, n):
        left[i] = left[i-1] * nums[i-1]
    for i in range(n-2, -1, -1):
        right[i] = right[i+1] * nums[i+1]

    return [left[i] * right[i] for i in range(n)]

# nums = [1,2,3,4] => [24, 12, 8, 6]
```

Solution 2 — $O(1)$ extra space (optimal):

```
Python

def product_except_self_v2(nums):
    n = len(nums)

    output = [1] * n

    for i in range(1, n):          # left prefix into output
        output[i] = output[i-1] * nums[i-1]

    right = 1

    for i in range(n-1, -1, -1):  # multiply right suffix in-place
        output[i] *= right
```

Solution	Time	Space	Key Idea
V1 — Two Arrays	$O(n)$	$O(n)$	Explicit left[] and right[] arrays
V2 — In-place	$O(n)$	$O(1)$	Reuse output array as left prefix store

Problem B — Set Matrix Zeroes

Task: If `matrix[i][j] == 0`, set its entire row and entire column to 0. Must be done in-place.

Numerical Example

Before			After		
1	1	1	1	0	1
1	0	1	0	0	0
1	1	1	1	0	1

Zero at position (1,1) zeroes the entire row 1 and column 1.

Solution 1 — HashSets $O(m+n)$ space:

```
def set_zeroes_v1(matrix):
    zero_rows, zero_cols = set(), set()
    for i in range(len(matrix)):
        for j in range(len(matrix[0])):
            if matrix[i][j] == 0:
                zero_rows.add(i)
                zero_cols.add(j)

    for i in range(len(matrix)):
        for j in range(len(matrix[0])):
            if i in zero_rows or j in zero_cols:
                matrix[i][j] = 0
```

Solution 2 — First row/column as markers, $O(1)$ space (optimal):

```
def set_zeroes_v2(matrix):  
    m, n = len(matrix), len(matrix[0])  
    for i in range(1, m):          # use first row/col as markers  
        for j in range(1, n):  
            if matrix[i][j] == 0:  
                matrix[i][0] = 0  
                matrix[0][j] = 0      # apply markers to interior  
            if matrix[i][0] == 0 or matrix[0][j] == 0:  
                matrix[i][j] = 0  
  
    if r0:                          # handle first row/col last  
        for j in range(n): matrix[0][j] = 0  
  
    if c0:
```

INFO

Common Pattern: Both problems share the same insight — the optimal $O(1)$ -space solution reuses an existing part of the data structure (output array or first row/column) as implicit scratch storage.